Name of the Student	**S. Yuva Guru Mani Varma**
Reg. No	**192425005**
GitHub Link	**https://github.com/YuvaGuruManiVarmaS-018/MLA0405-192425005**

SIMATS ENGINEERING

CO4 - ASSESSMENT TOOL 1

Design and Develop Model

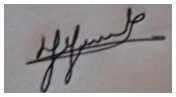
COURSE NAME: Deep Learning **COURSE CODE: MLA04**
SLOT : D

COURSE OUTCOME COVERED

CO 4: Students will be able to understand and apply probabilistic graphical models including Bayesian Networks, Markov Random Fields, Hidden Markov Models, and Conditional Random Fields. They will learn how to represent complex probabilistic relationships among variables and perform inference and learning in graphical models. Students will be able to use these models for reasoning under uncertainty and solving real-world decision-making problems. BL-6

Course Outcome Weightage: 10% **Duration: 90 minutes**
Assessment Tool Weightage: 30% **Mark : 25**

Code of Conduct:
I **S. Yuva Guru Mani Varma / 192425005** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student: **Name of the student: S. Yuva Guru Mani Varma**

Criteria	Problem Understanding & Requirement Analysis **(2)**	Model Architecture & Design **(2.5)**	Application of Deep Learning Techniques **(2.5)**	Model Development & Implementation Strategy **(2)**	Performance Analysis & Justification ***(1)***	**Total (10)**
Weightage	***20 %***	***30%***	***25%***	***15 %***	***10 %***	**100 %**
Q1						
Q2						
Q3						
Q4						
Q5						
Total						

QUESTIONS: 05 Total Marks:25

1.Desing and develop a model for a hospital has only **2,000 labeled X-ray images** for classifying images into normal and abnormal categories.

ANSWER:

1. CNN-Based Transfer Learning Model for X-Ray Classification

Since only **2,000 labeled X-ray images** are available, training a CNN completely from scratch may cause overfitting. A pre-trained **ResNet** can be used to transfer knowledge learned from a large image dataset.

Proposed Architecture

Input X-ray Image → Pre-trained ResNet → Global Average Pooling → New Fully Connected Layer → Softmax

Layer Selection

1. **Input Layer**
   - Resize X-ray images to the input size required by ResNet, such as **224 × 224 pixels**.
   - Normalize pixel values.
2. **Pre-trained Res Net Convolutional Layers**
   - Initially **freeze all convolutional layers**.
   - These layers contain useful low-level and mid-level features such as edges, textures, and shapes.
3. **Replace the Original Classification Layer**
   - Remove the original ResNet fully connected/classification layer.
   - Add:
     - Global Average Pooling
     - Dropout layer, e.g. **0.5**
     - Fully connected layer with **2 output neurons**
     - Softmax activation for **Normal** and **Abnormal** classes.
4. **Fine-Tuning**
   - First train only the newly added classification layers.
   - After the classifier becomes stable, **unfreeze the last one or two ResNet blocks**.
   - Train them using a **very small learning rate**, such as 1e-5.
   - Keep the earlier layers frozen because their general visual features are still useful.

Training Process

Stage	Frozen Layers	Trainable Layers
Stage 1	All ResNet layers	New classification layers
Stage 2	Early ResNet layers	Last ResNet block + new classifier

Use **data augmentation** such as rotation, scaling, cropping and horizontal flipping where medically appropriate. Use **binary cross-entropy** or two-class categorical cross-entropy depending on the output design.

Justification

Transfer learning reduces training time and the amount of data required. Freezing most ResNet layers prevents overfitting on the small 2,000-image dataset, while fine-tuning the final layers allows the network to learn X-ray-specific features. Dropout and augmentation further improve generalization.

Final model:

X-ray → ResNet feature extractor → GAP → Dropout → Dense(2) → Softmax → Normal/Abnormal

2. An agricultural organization wants to identify **five types of plant diseases** from leaf images. Since the dataset contains limited training samples, the organization wants to reuse features efficiently. **Design a Dense Net-based classification model** and explain how dense connections can help the model reuse features during classification.

ANSWER:

2. DenseNet-Based Plant Disease Classification Model

For classifying **five plant diseases** with limited training data, **DenseNet** is suitable because its dense connections allow features to be reused throughout the network.

Proposed Architecture

Leaf Image → DenseNet Backbone → Global Average Pooling → Dropout → Dense(5) → Softmax

Model Design

1. **Input**
   - Resize leaf images, for example, to **224 × 224 pixels**.
   - Normalize the images.
   - Apply augmentation such as rotation, zoom, shifting and flipping.
2. **Pre-trained DenseNet**
   - Use a pre-trained **DenseNet**, such as DenseNet-121.
   - Initially freeze most of its convolutional layers.
   - The pre-trained layers extract features such as edges, textures, leaf shapes and disease patterns.
3. **Replace Classification Layer**
   - Remove the original classification layer.

- Add:
  - Global Average Pooling
  - Dropout
  - Dense layer with **5 neurons**
  - Softmax activation.

4. **Fine-Tuning**
   - Train the new classification layer first.
   - Then unfreeze the final DenseNet block.
   - Fine-tune it with a small learning rate.

How Dense Connections Work

In a normal CNN, each layer mainly receives information from the previous layer.

In DenseNet:

Layer 1 → Layer 2 → Layer 3 → Layer 4

but each layer receives the outputs of **all previous layers**:

Layer 1 → Layer 2
Layer 1 + Layer 2 → Layer 3
Layer 1 + Layer 2 + Layer 3 → Layer 4

Thus, earlier features are directly available to later layers.

Advantages for Plant Disease Detection

- **Feature reuse:** Earlier features such as edges and textures can be reused by deeper layers.
- **Better gradient flow:** Dense connections help gradients propagate through the network.
- **Efficient learning:** The model does not need to relearn features at every layer.
- **Good performance with limited data:** Pre-trained DenseNet reduces the amount of training data required.
- **Disease-specific features:** Fine-tuning allows the network to learn spots, discoloration, lesions and other disease characteristics.

Final Model

Leaf Image → Pre-trained DenseNet → Feature Reuse through Dense Blocks → GAP → Dropout → Dense(5) → Softmax → Five Disease Classes

Conclusion: DenseNet is particularly appropriate for this problem because its dense connections promote **feature reuse and efficient gradient propagation**, while transfer learning reduces overfitting when only a limited number of plant images are available.

3. A university wants to predict a student's future academic performance using their **weekly attendance, assignment marks, internal assessment marks, and previous examination scores** collected over several weeks. **Design an LSTM-based model** that can process this sequential information and predict the student's performance category.

ANSWER:

LSTM-Based Model for Student Performance Prediction

Since student data is collected **over several weeks**, it forms sequential/time-series data. An **LSTM (Long Short-Term Memory)** network can learn patterns from previous weeks and predict the student's final performance category.

Model Architecture

Input Data → Preprocessing → LSTM Layer → Dropout → Dense Layer → Softmax → Performance Category

Processing Stages

1. **Data Collection**
    - Collect weekly values of:
        - Attendance
        - Assignment marks
        - Internal assessment marks
        - Previous examination scores
2. **Data Preprocessing**
    - Handle missing values.
    - Normalize the numerical values.
    - Arrange the data chronologically.
    - Create sequences such as:
      Week 1 → Week 2 → Week 3 → ... → Week N
3. **Input Layer**
    - Each time step contains four features:
      [Attendance, Assignment, Internal Assessment, Previous Exam Score]
4. **LSTM Layer**
    - The LSTM processes the weekly sequence.
    - It uses memory cells, input gates, forget gates, and output gates to retain important information from previous weeks.
    - This helps identify trends such as consistently improving or declining performance.
5. **Dropout Layer**
    - Dropout can be added to reduce overfitting.
6. **Dense Output Layer**
    - The LSTM output is passed to a fully connected layer.
    - For example, three output classes can be used:
        - **High Performance**
        - **Average Performance**
        - **Low Performance**
7. **Softmax Activation**
    - Softmax converts the outputs into probabilities.
    - The category with the highest probability is selected as the predicted performance category.

Example

If a student's weekly sequence is:

Week 1 → Week 2 → Week 3 → Week 4

and attendance and marks consistently decrease, the LSTM can learn this temporal pattern and predict:

Low Performance

Advantages

- Handles sequential student data effectively.
- Remembers important information from previous weeks.
- Captures performance trends over time.
- Suitable for predicting future student performance.
- Dropout helps reduce overfitting.

4. An organization wants to classify customer reviews as **positive, negative, or neutral**. Since the meaning of a word may depend on both the words appearing before and after it, **design a Bidirectional RNN model** for this application. Identify the major processing stages from input text to final sentiment prediction.

ANSWER:

Bidirectional RNN for Customer Sentiment Classification

A Bidirectional RNN (BiRNN) is suitable because the meaning of a word can depend on both the previous and following words in a review.

Model Architecture

Customer Review → Tokenization → Embedding → Bidirectional RNN → Dense Layer → Softmax → Sentiment

Major Processing Stages

1. **Input Text**
    - Example:
      "The movie was not very good."
2. **Text Preprocessing**
    - Convert text to lowercase.
    - Remove unnecessary symbols.
    - Tokenize the review into individual words.
    - Convert words into numerical token IDs.
3. **Word Embedding**
    - Convert each token into a dense numerical vector.
    - Similar words can have similar vector representations.
4. **Bidirectional RNN**
    - The review is processed in **two directions**:
        - Forward RNN: reads from **beginning → end**
        - Backward RNN: reads from **end → beginning**
    - Their outputs are combined.

This allows the model to understand both **past and future context**.

5. **Dense Layer**
    - The combined representation is passed to a fully connected layer.
    - It learns features useful for sentiment classification.
6. **Softmax Output**
    - The final layer contains **3 neurons**:
        - Positive
        - Negative
        - Neutral
    - Softmax produces the probability for each class.

Example

For the sentence:

"The movie was good but the ending was disappointing."

The Bidirectional RNN can consider words occurring **before and after** terms such as *good* and *disappointing*. This gives it better contextual understanding than a one-directional RNN.

Final Flow

Review → Preprocessing → Tokenization → Word Embedding → Forward RNN + Backward RNN → Combined Features → Dense Layer → Softmax → Positive/Negative/Neutral

Advantages

- Uses context from both directions.
- Better understanding of word relationships.
- Useful for reviews where sentiment depends on surrounding words.
- Provides improved sentiment classification compared with a simple one-directional RNN.

5. A software company wants to develop a system that translates **English sentences into Hindi** . The length of the input and output sentences may differ, and the system needs to understand the complete input sentence before generating the translated sentence. **Design an Encoder–Decoder model** and describe the role of the encoder, decoder, hidden representation, and output layer in the translation process.

ANSWER:

Encoder–Decoder Model for English-to-Hindi Translation

An **Encoder–Decoder architecture** is suitable for machine translation because the English input and Hindi output can have **different sentence lengths**. The encoder first understands the complete English sentence, and the decoder then generates the Hindi translation word by word.

Model Architecture

English Sentence → Encoder → Hidden Representation → Decoder → Softmax → Hindi Sentence

1. Encoder

The **encoder** reads the English sentence sequentially, word by word.

Example:

"I am going to school"

The encoder processes:

I → am → going → to → school

It converts the words into numerical representations using an **embedding layer** and processes them using an **LSTM/GRU/RNN**.

The encoder captures the important information and meaning of the complete input sentence.

2. Hidden Representation

After processing the complete English sentence, the encoder produces a **hidden representation (context vector)**.

This representation contains important information about:

- Meaning of the sentence
- Word relationships
- Sentence context
- Relevant information needed for translation

It acts as the information passed from the encoder to the decoder.

3. Decoder

The **decoder** receives the encoder's hidden representation and generates the Hindi sentence **one word at a time**.

For example:

English: "I am going to school."

Hindi: "मैं स्कूल जा रहा हूँ।"

The decoder starts with a special **start token <START>** and predicts the first Hindi word. It then uses the previously generated word and its hidden state to predict the next word.

The process continues until the **<END>** token is generated.

4. Output Layer

At every decoding step, the output layer produces probabilities for all possible Hindi words.

A **Softmax layer** is commonly used.

For example:

Hindi Word	Probability
मैं	0.80
वह	0.10
हम	0.05
अन्य	0.05

The word with the highest probability is selected as the next output word.

Training Process

During training, English-Hindi sentence pairs are provided:

English sentence → Correct Hindi sentence

The model learns to predict the correct Hindi word at each decoding step. **Cross-entropy loss** can be used to compare the predicted words with the actual Hindi words.

Role of Each Component

Component	Role
Encoder	Reads and understands the complete English sentence
Hidden Representation	Stores the important contextual information from the input
Decoder	Generates the Hindi translation sequentially
Output Layer	Produces probabilities for the next Hindi word using Softmax

Final Flow

English Input
↓
Embedding
↓
Encoder (LSTM/GRU)
↓
Hidden Representation / Context

↓
Decoder (LSTM/GRU)
↓
Dense + Softmax Output Layer
↓
Hindi Words → Complete Hindi Sentence

Conclusion

The Encoder–Decoder model solves the problem of different input and output sentence lengths by separating **sentence understanding** from **sentence generation**. The encoder converts the complete English sentence into a meaningful hidden representation, while the decoder uses this representation to generate the Hindi translation word by word until the end token is reached.

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Problem Understanding & Requirement Analysis	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Model Architecture & Design	30%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Application of Deep Learning Techniques	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Model Development & Implementation Strategy	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Performance Analysis & Justification	10%	Thorough analysis with validated results and	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

		performance evaluation			